

# CS 3307 Project Proposal

## Single Player Rhythm Game Engine

Group Number: 75

Seongmin Park  
spark776@uwo.ca  
251305418

# 1 Problem Statement

Rhythm games are a genre of the game where players need to respond to musical patterns by performing inputs at the correct time. Although the basic interaction (pressing a key when a note reaches a target) is simple, the game must coordinate timing, note patterns, scoring, combo tracking, difficulty, progression, and persistent player data. Small errors in these systems can produce inconsistent gameplay, such as notes being judged differently depending on timing, scores being calculated incorrectly, or player progress being lost. The proposed system is a desktop rhythm game that provides a structured environment for playing songs, receiving timing-based judgments, and tracking progression.

The primary users are players, who select songs and difficulties, play through the charts, and view their results. A player should be able to understand how well they performed through information such as score, accuracy, judgement counts, and combo.

A secondary stakeholder is the developer, who needs the game to have well-structured domain model so that scoring rules, difficulty configurations, and other gameplay variations can be added without substantially modifying the existing components. The project also needs to be maintainable and testable so that changes to one part of the system do not unexpectedly break other parts.

The main difficulty is that a rhythm game involves several components that must cooperate while remaining independent. For instance, gameplay depends on precise timing, but timing should not be tied directly to user interface or a particular timing implementation. Similarly, judging a note, calculating a score, maintaining a combo, and determining progression are different responsibilities that should not be concentrated in one large game class.

## **2 Requirements**

The initial requirements were gathered from three sources. First, the project's required Design Depth Profile and technical requirements were used to identify the architectural and domain capabilities that the system must support.

Second, multiple rhythm games that already exist were examined to identify common gameplay concepts such as song selection, note charts, timing judgements, combos, scoring, and results. Examples considered include well-known rhythm games such as *osu!mania* and *DJMAX RESPECT V*.

Finally, the proposed gameplay was considered from the perspective of a typical player to identify the actions that the user must be able to perform.

### **2.1 Function Requirements**

#### **2.1.1 FR1**

Display a list of available songs and select a song to play. Must have.

#### **2.1.2 FR2**

Select an available difficulty for a selected song. Must have.

#### **2.1.3 FR3**

Load the selected song's chart and present its notes to the player during gameplay. Must have.

#### **2.1.4 FR4**

Accept player input for a note and determine a timing judgement based on difference between the player's input time and the note's scheduled (intended) time. Must have.

#### **2.1.5 FR5**

Support 2 types of notes: tap, and hold notes. Each type must apply its appropriate gameplay behaviour when played. Must have.

#### **2.1.6 FR6**

Calculate and display the player's score, combo, and accuracy during or after a song based on player's note judgements. Must have.

#### **2.1.7 FR7**

Allow player to pause, resume, and complete an active game session, while rejecting invalid game states. Must have.

#### **2.1.8 FR8**

Save player's progression and song results and restore them everytime the application is opened. Must have.

#### **2.1.9 FR9**

Different game modes using different controls. Such as keyboard-based modes, and game mode that you control only with the mouse. Could have.

#### **2.1.10 FR10**

Unlock songs or difficulties when the player's progression satisfies the unlock conditions such as, but not limited to, achieve certain threshold of accuracy, or play a song certain times. Note that this will require to modify FR2 to add functionality that prevent player to choose locked songs/difficulties depending on their progression. Should have.

#### **2.1.11 FR11**

The system shall display an appropriate error message and preserve the existing valid state when a save file is missing or corrupted. Must have.

## **2.2 Non-functional Requirements**

### **2.2.1 NFR1 - Performance**

The system shall process a valid gameplay input and produce its corresponding domain judgement without noticeable delay during normal gameplay. Under normal single player gameplay conditions, the domain operation for processing a note input should complete within 50 ms.

### **2.2.2 NFR2 - Reliability**

The system shall not terminate/crash unexpectedly as a result of invalid player input, illegal game state transitions, or a corrupt save file. These conditions shall produce a defined error messages instead.

### **2.2.3 NFR3 - Maintainability**

The domain logic shall remain independent of Qt user interface classes. Domain classes shall be compilable and testable without the user interface part of the application.

### **2.2.4 NFR4 - Usability**

A player shall be able to start a song from the selection screen using the available controls without needing to configure files or interact with the application's underlying data files.

## **3 Use Case Model**

### **3.1 Use Cases**

#### **3.1.1 UC1 - Select Song**

The player views the available songs and selects a song to play.

#### **3.1.2 UC2 - Select Difficulty**

The player selects an available difficulty for the chosen song.

#### **3.1.3 UC3 - Select Game Mode**

The player selects a supported game mode and its associated input controls.

#### **3.1.4 UC4 - Play Song**

The player provides inputs while notes appear, and the system judges the inputs and updates the player's performance.

#### **3.1.5 UC5 - Pause/Resume Game**

The player pauses or resumes an active game session.

#### **3.1.6 UC6 - Complete Song**

The system detects the end of a song, calculates the final performance, and records the song result.

#### **3.1.7 UC7 - View Result**

The player views their final score, combo, accuracy, and note judgements after completing a song.

#### **3.1.8 UC8 - Save Progress**

The system saves the player's progression and song results to persistent storage.

#### **3.1.9 UC9 - Load Progress**

The system restores the player's saved progression and results when the application starts.

### 3.1.10 UC10 - Unlock Content

The system evaluates the player's progression and unlocks songs or difficulties when their unlock conditions are satisfied.

### 3.1.11 UC11 - Handle Invalid Save

The system detects a missing or corrupted save file, informs the player, and preserves or creates a valid profile state.

## 3.2 Traceability Table

| Functional Requirements | Realized Use Case(s) |
|-------------------------|----------------------|
| FR1                     | UC1                  |
| FR2                     | UC2, UC10            |
| FR3                     | UC1, UC4             |
| FR4                     | UC4                  |
| FR5                     | UC4                  |
| FR6                     | UC4, UC6, UC7        |
| FR7                     | UC4, UC5, UC6        |
| FR8                     | UC8, UC9             |
| FR9                     | UC3, UC4             |
| FR10                    | UC2, UC10            |
| FR11                    | UC9, UC11            |

### 3.3 UML Use Case Diagram

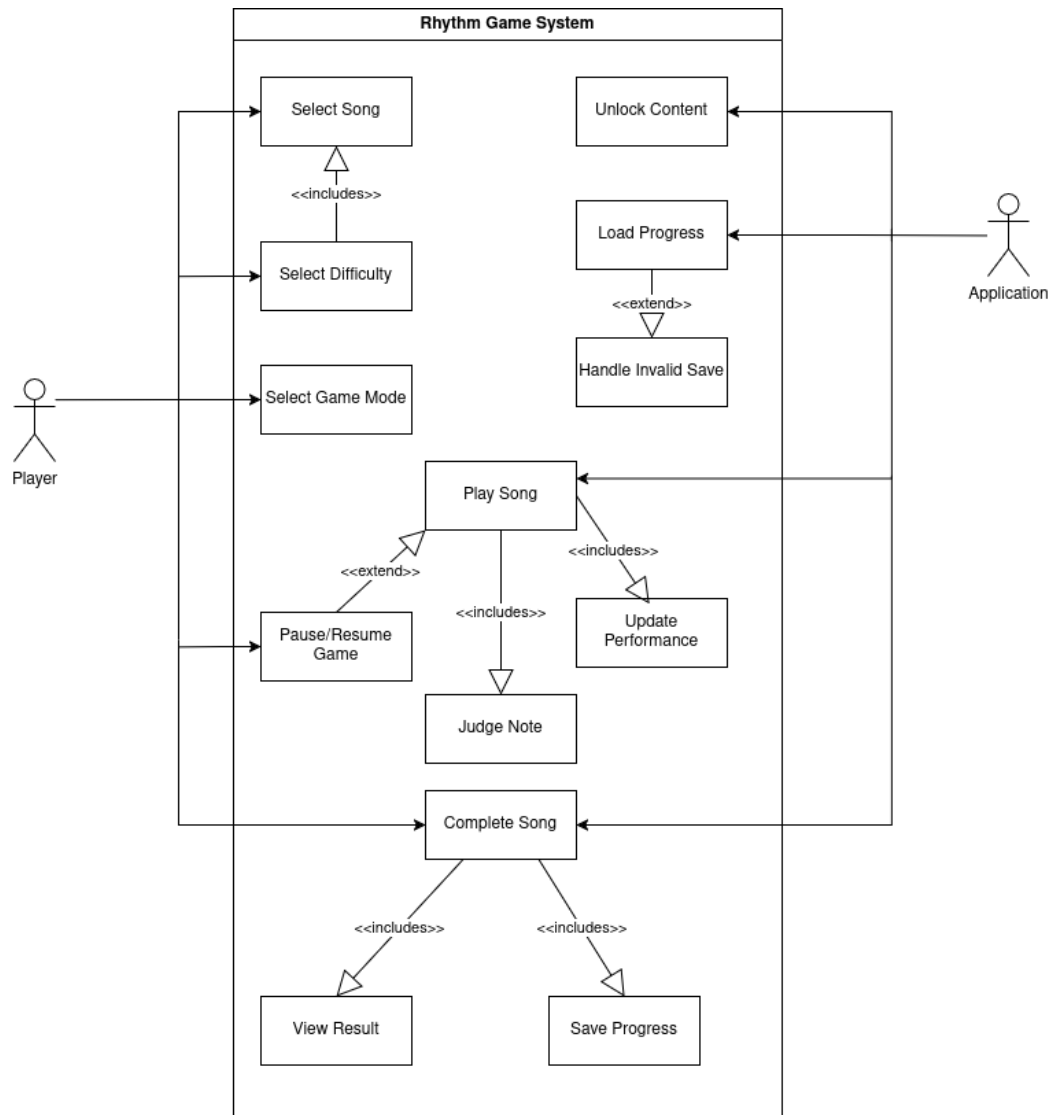


Figure 1: Use Case Diagram

## **3.4 Written Use Case - UC4**

### **3.4.1 Primary Actor**

Player

### **3.4.2 Goal**

Play a selected song and receive timing judgements and performance feedback based on their inputs.

### **3.4.3 Preconditions**

1. The application is running.
2. The player has selected a valid song.
3. The player has selected an available difficulty.
4. The selected chart can be loaded successfully.
5. A valid game mode has been selected.
6. A 'GameSession' can be created in the initial state.

### **3.4.4 Main Success Scenario**

1. The player selects Play from the song selection screen.
2. The system loads the selected chart.
3. The system creates a new game session using selected song, difficulty, and game mode.
4. The game session transitions from 'Loading' to 'Playing'.
5. The system begins presenting notes according to their scheduled times.
6. The player provides an input corresponding to an upcoming note.
7. The system obtains the input timestamp.
8. The system identifies the relevant note.
9. The system calculates the difference between the player's input time and the note's scheduled time.

10. The system determines a timing judgement, such as 'Perfect', 'Great', 'Good', or 'Miss'.
11. The system applies the appropriate behaviour for the note type. For example, a tap note is resolved by a single input, while a hold note requires the player to maintain the input for its required duration.
12. The system updates the player's score, combo, and accuracy.
13. The updated performance information is displayed to the player.
14. Steps 6-13 repeat as the remaining notes are played.
15. When the chart is complete, the system transitions the game session to 'Finished'.
16. The system creates a song result containing the player's score, accuracy, combo, and judgement information.
17. The result is displayed to the player.
18. The player's progression and result are saved.

#### **3.4.5 Alternative Flows**

- A1: Player pauses the game.
  1. While the session is 'Playing', the player selects pause.
  2. The system transitions the session to 'Paused'.
  3. Note progression and gameplay input processing stop.
  4. The player selects resume.
  5. The system transitions the session back to 'Playing'.
  6. Gameplay continues from the paused state.
- A2: Player completes a hold note.
  1. The player begins the hold input within the valid timing window.
  2. The system records the start of the hold.
  3. The player maintains the input for the required duration.
  4. The system validates the hold duration.
  5. The hold note is successfully resolved and the appropriate score is awarded.

- A3: Player makes a late or early input.
  1. The player's input occurs outside the 'Perfect' timing window.
  2. The system calculates the timing difference.
  3. The appropriate lower judgement is assigned.
  4. The score, combo, and accuracy are updated according to the judgement.

#### **3.4.6 Exception Flows**

- E1: Invalid player input
  1. The player provides an input that does not correspond to a valid control or lane.
  2. The system rejects the input.
  3. The system displays an appropriate feedback message where applicable.
  4. The current game state, score, and valid notes remain unchanged.
- E2: Illegal game state transition
  1. An attempt is made to perform an operation that is invalid for the current state, such as resuming a session that is already 'Playing'.
  2. The system rejects the transition.
  3. The session remains in its current valid state.
  4. The system reports the error appropriately.
- E3: Chart cannot be loaded
  1. The selected chart cannot be loaded or is invalid.
  2. The system does not start the game session.
  3. The player is informed that the song cannot currently be played.
  4. The player remains on the song selection screen.

#### **3.4.7 Postconditions**

- Successful Completion
  1. The game session is in the 'Finished' state.

2. Every required note has been resolved.
  3. A song result containing the player's performance has been created.
  4. Score, combo, accuracy, and judgement information have been calculated.
  5. The result is displayed to the player.
  6. The player's progression is updated.
  7. The updated progression is saved.
- Unsuccessful Completion
    1. The game session remains in a valid state or is not created.
    2. No invalid score or progression data is stored.
    3. The player receives appropriate feedback about the failure.

## 4 Initial Class Diagram

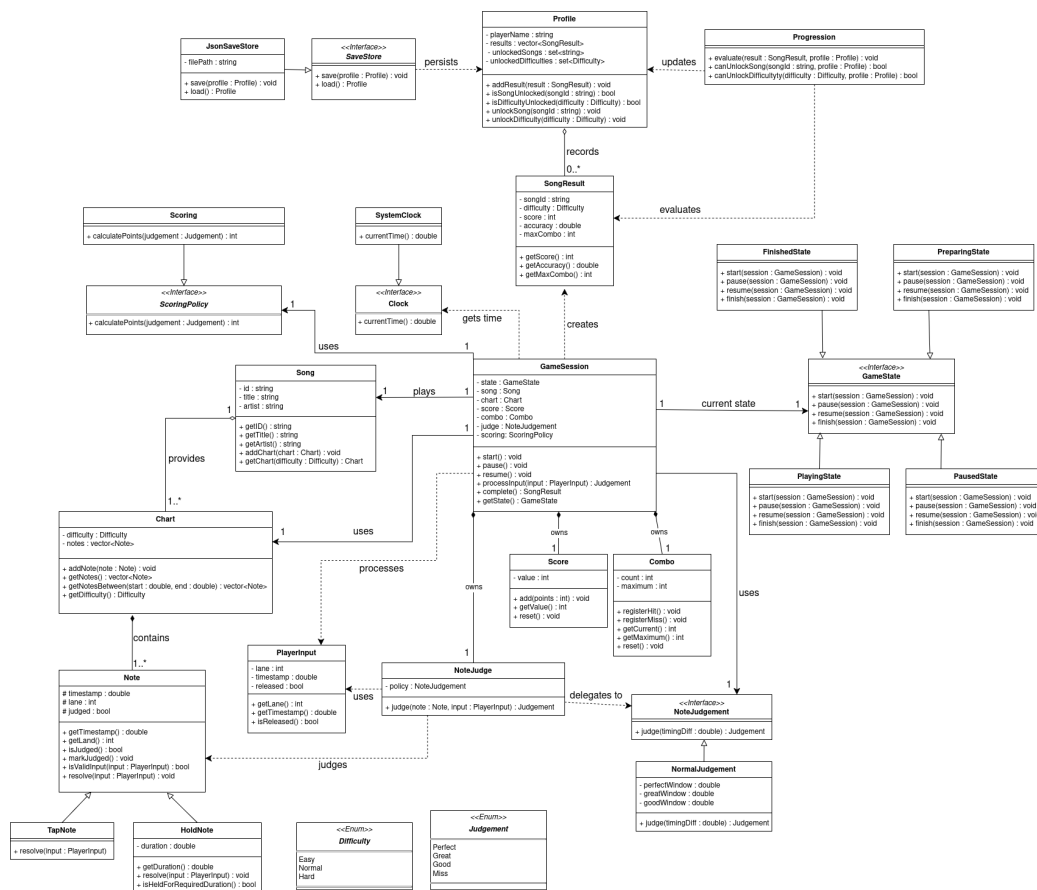


Figure 2: Initial Class Diagram

Since the diagram is big, you may not clearly see everything on this document.

If that's the case, see [image on github](#) for details.

## 5 Sequence Diagram

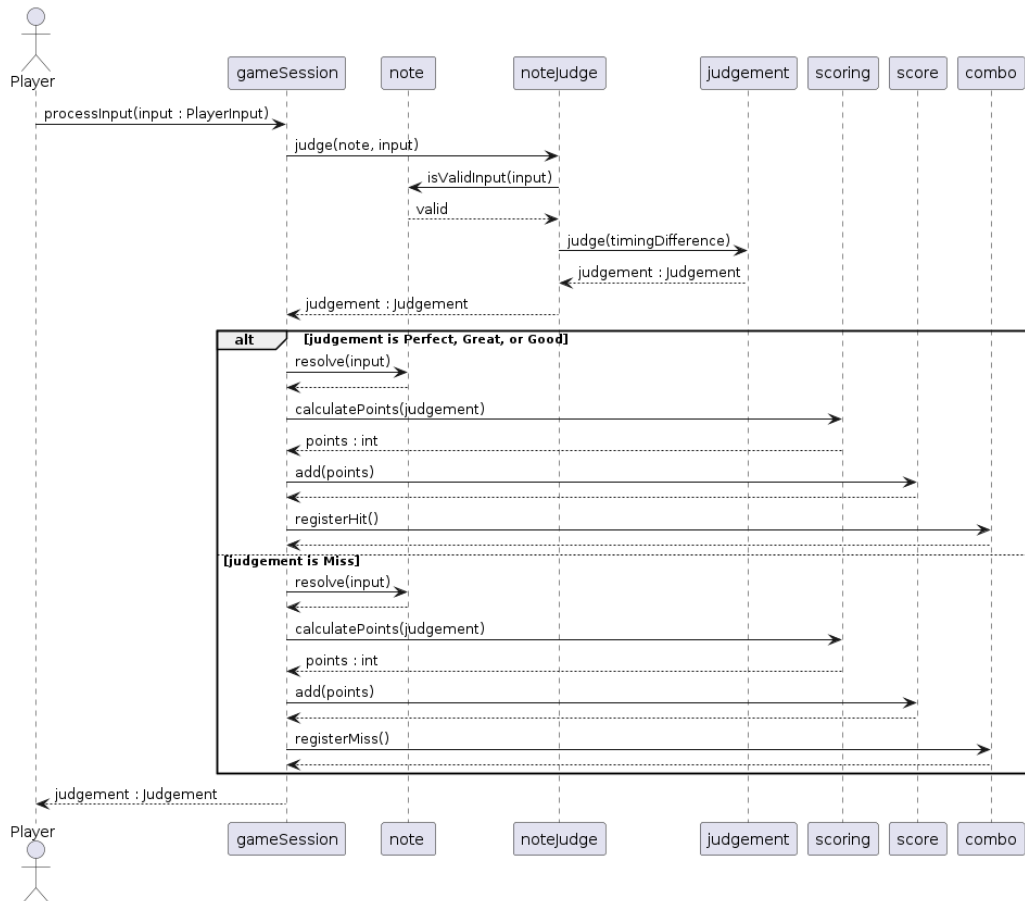


Figure 3: Sequence Diagram

If you can't read some texts, see [image on github](#).

- 6 Architecture Overview
- 7 Object-Oriented Design Approach & Candidate Patterns
- 8 Milestone Plans & Risks
- 9 Design Depth Profile